Architecture 1001:
x86-64 Assembly
READ THE *FUN* MANUAL!

Xeno Kovah – 2021
xeno@darkmentor.com

All materials is licensed under a Creative Commons “Share Alike” license.

- http://creativecommons.org/licenses/by-sa/4.0/

You are free:

to Share — to copy, distribute and transmit the work

to Remix — to adapt the work

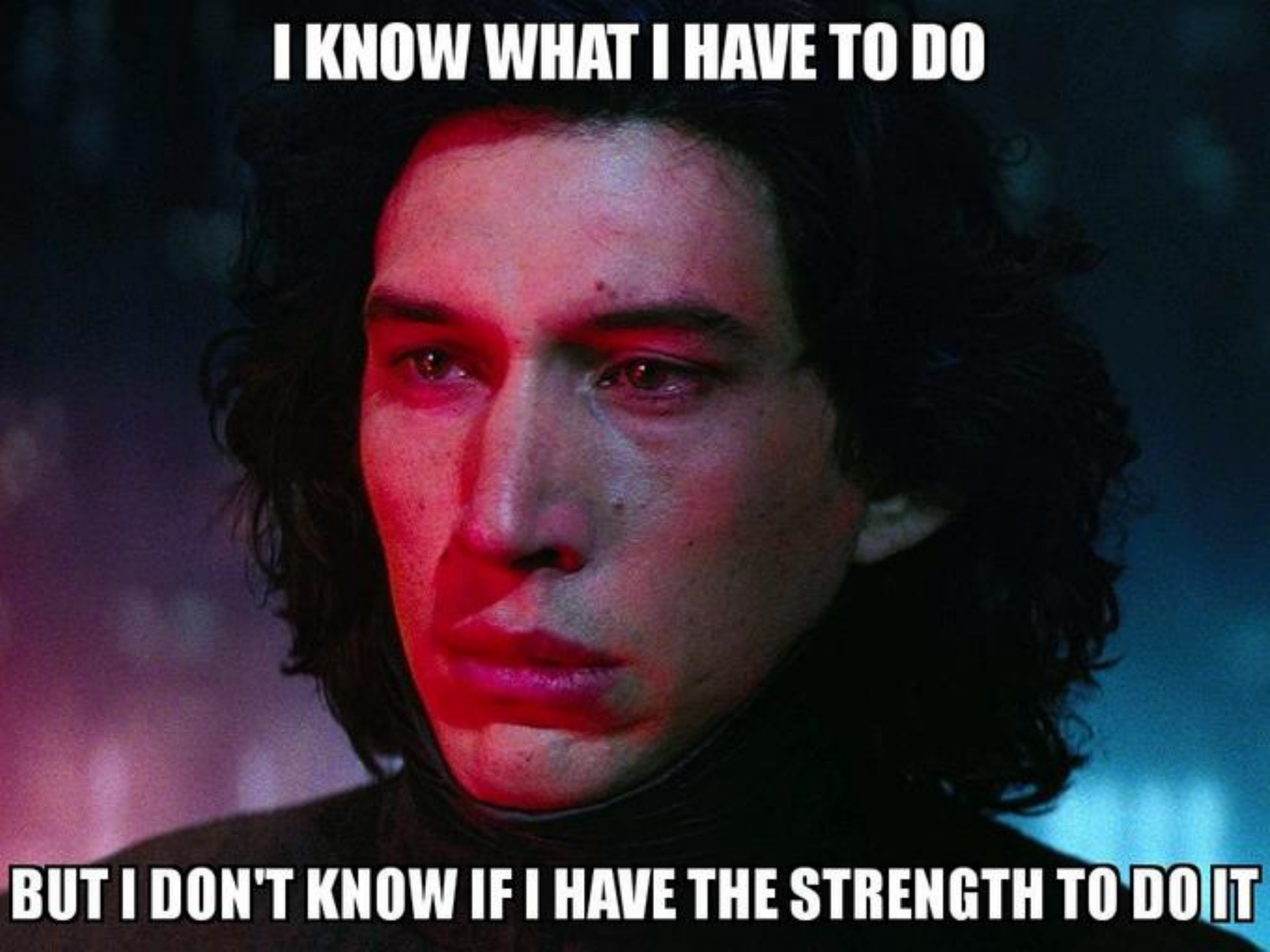
Under the following conditions:

Attribution — You must attribute the work in the manner specified by the author or licensor (but not in any way that suggests that they endorse you or your use of the work).

Share Alike — If you alter, transform, or build upon this work, you may distribute the resulting work only under the same, similar or a compatible license.

Attribution condition: You must indicate that derivative work
"Is derived from Xeno Kovah's 'Architecture 1001: x86-64 Assembly’ class, available at https://ost.fyi"

I KNOW WHAT I HAVE TO DO



BUT I DON'T KNOW IF I HAVE THE STRENGTH TO DO IT

Guess what?
I have repeatedly misled you!

- Simplification is misleading
- Time to learn the *fascinating* truth…
- Time to RTFM!

Read The *Fun* Manuals

- http://www.intel.com/products/processor/manuals/
- Vol.1 is a summary of life, the universe, and everything about x86
- Vol. 2a-d explains all the instructions
- Vol. 3a-d are all the gory details for all the extra stuff they've added in over the years (MultiMedia eXtentions - MMX, Virtual Machine eXtentions - VMX, virtual memory, 16/64 bit modes, system management mode, etc)
- Reminder to use the pre-downloaded *Nov 2020* version of the manual which we've been using as the standardized reference throughout this class, so we're all looking at the same information
- We're primarily looking at Vol. 2 in this class

Interpreting the Instruction Reference Pages

- The correct way to interpret these pages is given in the Intel Manual 2a, Sections 2.1 & 2.2
- I will give yet another simplification for now, and have a deeper dive be optional material
- Moral of the story is that you have to RTFM to RTFM ;)

AND - Bitwise AND

- C binary operator "&" (not &&, that's *logical* AND)
- Destination operand can be r/mX or register
- Source operand can be r/mX or register or immediate (No source *and* destination as r/mXs)

and al, bl

	00110011b (al - 0x33)
AND	01010101b (bl - 0x55)
result	00010001b (al - 0x11)

and al, 0x42

	00110011b (al - 0x33)
AND	01000010b (imm - 0x42)
result	00000010b (al - 0x02)

AND - Bitwise AND

- C binary operator "&" (not &&, that's *logical* AND)
- Destination operand can be r/mX or register
- Source operand can be r/mX or register or immediate (No s*I Said:* destination as r/mXs)

What

and al, bl

	00110011b (al - 0x33)
AND	01010101b (bl - 0x55)
result	00010001b (al - 0x11)

and al, 0x42

	00110011b (al - 0x33)
AND	01000010b (imm - 0x42)
result	00000010b (al - 0x02)

AND—Logical AND

Opcode	Instruction	Op/ En	64-bit Mode	Compat/ Leg Mode	Description
24 *ib*	AND AL, *imm8*	I	Valid	Valid	AL AND *imm8.*
25 *iw*	AND AX, *imm16*	I	Valid	Valid	AX AND i*mm16.*
25 *id*	AND EAX, *imm32*	I	Valid	Valid	EAX AND *imm32.*
REX.W + 25 *id*	AND RAX, *imm32*	I	Valid	N.E.	RAX AND *imm32 sign-extended to 64-bits.*
80 /4 *ib*	AND *r/m8, imm8*	MI	Valid	Valid	*r/m8* AND *imm8.*
REX + 80 /4 *ib*	AND *r/m8**, *imm8*	MI	Valid	N.E.	*r/m8* AND *imm8.*
81 /4 *iw*	AND *r/m16, imm16*	MI	Valid	Valid	*r/m16* AND *imm16.*
81 /4 *id*	AND *r/m32, imm32*	MI	Valid	Valid	*r/m32* AND *imm32.*
REX.W + 81 /4 *id*	AND *r/m64, imm32*	MI	Valid	N.E.	*r/m64* AND *imm32 sign extended to 64-bits.*
83 /4 *ib*	AND *r/m16, imm8*	MI	Valid	Valid	*r/m16* AND *imm8 (sign-extended).*
83 /4 *ib*	AND *r/m32, imm8*	MI	Valid	Valid	*r/m32* AND *imm8 (sign-extended).*
REX.W + 83 /4 *ib*	AND *r/m64, imm8*	MI	Valid	N.E.	*r/m64* AND *imm8 (sign-extended).*
20 /*r*	AND *r/m8, r8*	MR	Valid	Valid	*r/m8* AND *r8.*
REX + 20 /*r*	AND *r/m8**, *r8**	MR	Valid	N.E.	*r/m64* AND *r8 (sign-extended).*
21 /*r*	AND *r/m16, r16*	MR	Valid	Valid	*r/m16* AND *r16.*
21 /*r*	AND *r/m32, r32*	MR	Valid	Valid	*r/m32* AND *r32.*
REX.W + 21 /*r*	AND *r/m64, r64*	MR	Valid	N.E.	*r/m64* AND *r32.*
22 /*r*	AND *r8, r/m8*	RM	Valid	Valid	*r8* AND *r/m8.*
REX + 22 /*r*	AND *r8**, *r/m8**	RM	Valid	N.E.	*r/m64* AND *r8 (sign-extended).*
23 /*r*	AND *r16, r/m16*	RM	Valid	Valid	*r16* AND *r/m16.*
23 /*r*	AND *r32, r/m32*	RM	Valid	Valid	*r32* AND *r/m32.*
REX.W + 23 /*r*	AND *r64, r/m64*	RM	Valid	N.E.	*r64* AND *r/m64.*

AND—Logical AND

Opcode	Instruction	Op/ En	64-bit Mode	Compat/ Leg Mode	Description
24 *ib*	AND AL, *imm8*	I	Valid	Valid	AL AND *imm8.*
25 *iw*	AND AX, *imm16*	I	Valid	Valid	AX AND i*mm16.*
25 *id*	AND EAX, *imm32*	I	Valid	Valid	EAX AND *imm32.*
REX.W + 25 *id*	AND RAX, *imm32*	I	Valid	N.E.	RAX AND *imm32 sign-extended to 64-bits.*
80 /4 *ib*	AND *r/m8, imm8*			lid	*r/m8* AND *imm8.*
REX + 80 /4 *ib*	AND *r/m8**, *imm8*			E.	*r/m8* AND *imm8.*
81 /4 *iw*	AND *r/m16, imm16*			lid	*r/m16* AND *imm16.*
81 /4 *id*	AND *r/m32, imm32*			lid	*r/m32* AND *imm32.*
REX.W + 81 /4 *id*	AND *r/m64, imm32*			E.	*r/m64* AND *imm32 sign extended to 64-bits.*
83 /4 *ib*	AND *r/m16, imm8*			lid	*r/m16* AND *imm8 (sign-extended).*
83 /4 *ib*	AND *r/m32, imm8*			lid	*r/m32* AND *imm8 (sign-extended).*
REX.W + 83 /4 *ib*	AND *r/m64, imm8*			E.	*r/m64* AND *imm8 (sign-extended).*
20 /r	AND *r/m8, r8*			lid	*r/m8* AND *r8.*
REX + 20 /r	AND *r/m8**, *r8**			E.	*r/m64* AND *r8 (sign-extended).*
21 /r	AND *r/m16, r16*			lid	*r/m16* AND *r16.*
21 /r	AND *r/m32, r32*	MR	Valid	Valid	*r/m32* AND *r32.*
REX.W + 21 /r	AND *r/m64, r64*	MR	Valid	N.E.	*r/m64* AND *r32.*
22 /r	AND *r8, r/m8*	RM	Valid	Valid	*r8* AND *r/m8.*
REX + 22 /r	AND *r8**, *r/m8**	RM	Valid	N.E.	*r/m64* AND *r8 (sign-extended).*
23 /r	AND *r16, r/m16*	RM	Valid	Valid	*r16* AND *r/m16.*
23 /r	AND *r32, r/m32*	RM	Valid	Valid	*r32* AND *r/m32.*
REX.W + 23 /r	AND *r64, r/m64*	RM	Valid	N.E.	*r64* AND *r/m64.*

What The Manual Says :O

AND—Logical AND

Opcode	Instruction	Op/ En	64-bit Mode	Compat/ Leg Mode	Description
24 *ib*	AND AL, *imm8*	I	Valid	Valid	AL AND *imm8.*
25 *iw*	AND AX, *imm16*	I	Valid	Valid	AX AND i*mm16.*
25 *id*	AND EAX, *imm32*	I	Valid	Valid	EAX AND *imm32.*
REX.W + 25 *id*	AND RAX, *imm32*	I	Valid	N.E.	RAX AND *imm32 sign-extended to 64-bits.*

- Opcode Column
- Represents the literal byte value(s) which correspond to the given instruction
- In this case, if you were to see a 0x24 followed by a byte, or 0x25 followed by 4 bytes, you would know they were specific forms of the AND instruction

For more detail, see Intel Vol. 2a section 3.1.1.1
“Opcode Column in the Instruction Summary Table (Instructions without VEX Prefix)”

AND—Logical AND

Opcode	Instruction	Op/ En	64-bit Mode	Compat/ Leg Mode	Description
24 *ib*	AND AL, *imm8*	I	Valid	Valid	AL AND *imm8.*
25 *iw*	AND AX, *imm16*	I	Valid	Valid	AX AND i*mm16.*
25 *id*	AND EAX, *imm32*	I	Valid	Valid	EAX AND *imm32.*
REX.W + 25 *id*	AND RAX, *imm32*	I	Valid	N.E.	RAX AND *imm32 sign-extended to 64-bits.*

- If it was 0x25, how would you know whether it should be followed by 2 bytes (imm16) or 4 bytes (imm32)?
- The length of the operand depends on if the processor is in 16-bit, 32-bit, or 64-bit mode
- Each mode has a default operand size (i.e. the size of the value)

AND—Logical AND

Opcode	Instruction	Op/ En	64-bit Mode	Compat/ Leg Mode	Description
24 *ib*	AND AL, *imm8*	I	Valid	Valid	AL AND *imm8.*
25 *iw*	AND AX, *imm16*	I	Valid	Valid	AX AND i*mm16.*
25 *id*	AND EAX, *imm32*	I	Valid	Valid	EAX AND *imm32.*
REX.W + 25 *id*	AND RAX, *imm32*	I	Valid	N.E.	RAX AND *imm32 sign-extended to 64-bits.*

- For 64-bit mode, the default operand size is 32-bits for most instructions and the default address size is 64-bits
- This means the default interpretation will usually be the ones with the r/m32, r32, imm32, or in this case a specific register like EAX, unless explicitly overridden with special instruction prefixes
- These are talked about later in optional material

AND—Logical AND

Opcode	Instruction	Op/ En	64-bit Mode	Compat/ Leg Mode	Description
24 *ib*	AND AL, *imm8*	I	Valid	Valid	AL AND *imm8.*
25 *iw*	AND AX, *imm16*	I	Valid	Valid	AX AND i*mm16.*
25 *id*	AND EAX, *imm32*	I	Valid	Valid	EAX AND *imm32.*
REX.W + 25 *id*	AND RAX, *imm32*	I	Valid	N.E.	RAX AND *imm32 sign-extended to 64-bits.*

- Instruction Column
- The human-readable mnemonic which is used to represent the instruction.
- This will frequently contain special encodings such as the “r/mX format” which I’ve previously discussed

For more detail, see Intel Vol. 2a section 3.1.1.3
“Instruction Column in the Opcode Summary Table”

AND—Logical AND

Opcode	Instruction	Op/ En	64-bit Mode	Compat/ Leg Mode	Description
24 *ib*	AND AL, *imm8*	I	Valid	Valid	AL AND *imm8.*
25 *iw*	AND AX, *imm16*	I	Valid	Valid	AX AND i*mm16.*
25 *id*	AND EAX, *imm32*	I	Valid	Valid	EAX AND *imm32.*
REX.W + 25 *id*	AND RAX, *imm32*	I	Valid	N.E.	RAX AND *imm32 sign-extended to 64-bits.*

- "Operand Encoding" Column
- Indicates what forms the operands can take

Instruction Operand Encoding

Op/En	Operand 1	Operand 2	Operand 3	Operand 4
RM	ModRM:reg (r, w)	ModRM:r/m (r)	NA	NA
MR	ModRM:r/m (r, w)	ModRM:reg (r)	NA	NA
MI	ModRM:r/m (r, w)	imm8/16/32	NA	NA
I	AL/AX/EAX/RAX	imm8/16/32	NA	NA

For more detail, see Intel Vol. 2a section 3.1.1.4
“Operand Encoding Column in the Instruction Summary Table”

AND—Logical AND

Opcode	Instruction	Op/ En	64-bit Mode	Compat/ Leg Mode	Description
24 *ib*	AND AL, *imm8*	I	Valid	Valid	AL AND *imm8.*
25 *iw*	AND AX, *imm16*	I	Valid	Valid	AX AND i*mm16.*
25 *id*	AND EAX, *imm32*	I	Valid	Valid	EAX AND *imm32.*
REX.W + 25 *id*	AND RAX, *imm32*	I	Valid	N.E.	RAX AND *imm32 sign-extended to 64-bits.*

- 64bit Column
- Whether or not the opcode is valid in 64 bit mode

V — Supported.

I — Not supported.

Should say "Valid" and "Invalid"

N.E. — Indicates an instruction syntax is not encodable in 64-bit mode (it may represent part of a sequence of valid instructions in other modes).

N.P. — Indicates the REX prefix does not affect the legacy instruction in 64-bit mode.

N.I. — Indicates the opcode is treated as a new instruction in 64-bit mode.

N.S. — Indicates an instruction syntax that requires an address override prefix in 64-bit mode and is not supported. Using an address override prefix in 64-bit mode may result in model-specific execution behavior.

For more detail, see Intel Vol. 2a section 3.1.1.5
“64/32-bit Mode Column in the Instruction Summary Table”

AND—Logical AND

Opcode	Instruction	Op/ En	64-bit Mode	Compat/ Leg Mode	Description
24 *ib*	AND AL, *imm8*	I	Valid	Valid	AL AND *imm8.*
25 *iw*	AND AX, *imm16*	I	Valid	Valid	AX AND i*mm16.*
25 *id*	AND EAX, *imm32*	I	Valid	Valid	EAX AND *imm32.*
REX.W + 25 *id*	AND RAX, *imm32*	I	Valid	N.E.	RAX AND *imm32 sign-extended to 64-bits.*

- Compatibility/Legacy Mode Column
- Whether or not the opcode is valid in 32/16 bit code

V — Supported.

I — Not supported.

Should say "Valid" and "Invalid"

• **N.E. —** Indicates an Intel 64 instruction mnemonics/syntax that is not encodable; the opcode sequence is not applicable as an individual instruction in compatibility mode or IA-32 mode. The opcode may represent a valid sequence of legacy IA-32 instructions.

For more detail, see Intel Vol. 2a section 3.1.1.5
“64/32-bit Mode Column in the Instruction Summary Table”

AND—Logical AND

Opcode	Instruction	Op/ En	64-bit Mode	Compat/ Leg Mode	Description
24 *ib*	AND AL, *imm8*	I	Valid	Valid	AL AND *imm8.*
25 *iw*	AND AX, *imm16*	I	Valid	Valid	AX AND i*mm16.*
25 *id*	AND EAX, *imm32*	I	Valid	Valid	EAX AND *imm32.*
REX.W + 25 *id*	AND RAX, *imm32*	I	Valid	N.E.	RAX AND *imm32 sign-extended to 64-bits.*

- Description Column
- Simple description of the action performed by the instruction
- Typically this just conveys the flavor of the instruction, but the majority of the details are in the main description text

See Intel Vol. 2a section 3.1.1.7
“Description Column in the Instruction Summary Table”

AND—Logical AND

Opcode	Instruction	Op/ En	64-bit Mode	Compat/ Leg Mode	Description
24 *ib*	AND AL, *imm8*	I	Valid	Valid	AL AND *imm8.*
25 *iw*	AND AX, *imm16*	I	Valid	Valid	AX AND i*mm16.*
25 *id*	AND EAX, *imm32*	I	Valid	Valid	EAX AND *imm32.*
REX.W + 25 *id*	AND RAX, *imm32*	I	Valid	N.E.	RAX AND *imm32 sign-extended to 64-bits.*
80 /4 *ib*	AND *r/m8, imm8*	MI	Valid	Valid	*r/m8* AND *imm8.*
REX + 80 /4 *ib*	AND *r/m8**, *imm8*	MI	Valid	N.E.	*r/m8* AND *imm8.*
81 /4 *iw*	AND *r/m16, imm16*	MI	Valid	Valid	*r/m16* AND *imm16.*
81 /4 *id*	AND *r/m32, imm32*	MI	Valid	Valid	*r/m32* AND *imm32.*
REX.W + 81 /4 *id*	AND *r/m64, imm32*	MI	Valid	N.E.	*r/m64* AND *imm32 sign extended to 64-bits.*
83 /4 *ib*	AND *r/m16, imm8*	MI	Valid	Valid	*r/m16* AND *imm8 (sign-extended).*
83 /4 *ib*	AND *r/m32, imm8*	MI	Valid	Valid	*r/m32* AND *imm8 (sign-extended).*
REX.W + 83 /4 *ib*	AND *r/m64, imm8*	MI	Valid	N.E.	*r/m64* AND *imm8 (sign-extended).*
20 /*r*	AND *r/m8, r8*	MR	Valid	Valid	*r/m8* AND *r8.*
REX + 20 /*r*	AND *r/m8**, *r8**	MR	Valid	N.E.	*r/m64* AND *r8 (sign-extended).*
21 /*r*	AND *r/m16, r16*	MR	Valid	Valid	*r/m16* AND *r16.*
21 /*r*	AND *r/m32, r32*	MR	Valid	Valid	*r/m32* AND *r32.*
REX.W + 21 /*r*	AND *r/m64, r64*	MR	Valid	N.E.	*r/m64* AND *r32.*
22 /*r*	AND *r8, r/m8*	RM	Valid	Valid	*r8* AND *r/m8.*
REX + 22 /*r*	AND *r8**, *r/m8**	RM	Valid	N.E.	*r/m64* AND *r8 (sign-extended).*
23 /*r*	AND *r16, r/m16*	RM	Valid	Valid	*r16* AND *r/m16.*
23 /*r*	AND *r32, r/m32*	RM	Valid	Valid	*r32* AND *r/m32.*
REX.W + 23 /*r*	AND *r64, r/m64*	RM	Valid	N.E.	*r64* AND *r/m64.*

22 /r	AND *r8, r/m8*	RM	Valid	Valid	*r8* AND *r/m8.*
REX + *22 /r*	AND *r8**, *r/m8**	RM	Valid	N.E.	*r/m64* AND *r8 (sign-extended).*
23 /r	AND *r16, r/m16*	RM	Valid	Valid	*r16* AND *r/m16.*
23 /r	AND *r32, r/m32*	RM	Valid	Valid	*r32* AND *r/m32.*
REX.W + *23 /r*	AND *r64, r/m64*	RM	Valid	N.E.	*r64* AND *r/m64.*

22 /r	AND *r8, r/m8*	RM	Valid	Valid	*r8* AND *r/m8.*
REX + *22 /r*	AND *r8**, *r/m8**	RM	Valid	N.E.	*r/m64* AND *r8 (sign-extended).*
23 /r	AND *r16, r/m16*	RM	Valid	Valid	*r16* AND *r/m16.*
23 /r	AND *r32, r/m32*	RM	Valid	Valid	*r32* AND *r/m32.*
REX.W + *23 /r*	AND *r64, r/m64*	RM	Valid	N.E.	*r64* AND *r/m64.*

- Looking at some other forms, we now see those “r/mX” things I told you about
- The main thing you need to know is that any time you see a r/mX, it can be either a register or memory value.
- Looking at the opcode column, we can infer that /r is probably a register placeholder, but what is the REX or REX.W?
- As mentioned before, the in-depth understanding of the opcode column is in optional material

22 /r	AND *r8, r/m8*	RM	Valid	Valid	*r8* AND *r/m8.*
REX + 22 /r	AND *r8*, r/m8**	RM	Valid	N.E.	*r/m64* AND *r8 (sign-extended).*
23 /r	AND *r16, r/m16*	RM	Valid	Valid	*r16* AND *r/m16.*
23 /r	AND *r32, r/m32*	RM	Valid	Valid	*r32* AND *r/m32.*
REX.W + 23 /r	AND *r64, r/m64*	RM	Valid	N.E.	*r64* AND *r/m64.*



- Looking at some other forms, we now see those “r/mX” things I told you about
- The main thing you need to know is that any time you see a r/mX, it can be either a register or memory value.
- Looking at the opcode column, we can infer that /r is probably a register placeholder, but what is the REX or REX.W?
- As mentioned before, the in-depth understanding of the opcode column is in optional material

Which Description Is Buggy?

80 /4 *ib*	AND *r/m8, imm8*	MI	Valid	Valid	*r/m8* AND *imm8.*
REX + 80 /4 *ib*	AND *r/m8**, *imm8*	MI	Valid	N.E.	*r/m8* AND *imm8.*
20 /r	AND *r/m8, r8*	MR	Valid	Valid	*r/m8* AND *r8.*
REX + 20 /r	AND *r/m8**, *r8**	MR	Valid	N.E.	*r/m64* AND *r8 (sign-extended).*
22 /r	AND *r8, r/m8*	RM	Valid	Valid	*r8* AND *r/m8.*
REX + 22 /r	AND *r8**, *r/m8**	RM	Valid	N.E.	*r/m64* AND *r8 (sign-extended).*

Which Description Is Buggy?

80 /4 *ib*	AND *r/m8, imm8*	MI	Valid	Valid	*r/m8* AND *imm8.*
REX + 80 /4 *ib*	AND *r/m8**, *imm8*	MI	Valid	N.E.	*r/m8* AND *imm8.*
20 /r	AND *r/m8, r8*	MR	Valid	Valid	*r/m8* AND *r8.*
REX + 20 /r	AND *r/m8**, *r8**	MR	Valid	N.E.	*r/m64* AND *r8 (sign-extended).*
22 /r	AND *r8, r/m8*	RM	Valid	Valid	*r8* AND *r/m8.*
REX + 22 /r	AND *r8**, *r/m8**	RM	Valid	N.E.	*r/m64* AND *r8 (sign-extended).*

80 /0 *ib*	ADD *r/m8, imm8*	MI	Valid	Valid	Add *imm8* to *r/m8.*
REX + 80 /0 *ib*	ADD *r/m8**, *imm8*	MI	Valid	N.E.	Add *sign*-extended *imm8* to *r/m8.*
00 /r	ADD *r/m8, r8*	MR	Valid	Valid	Add *r8* to *r/m8.*
REX + 00 /r	ADD *r/m8**, *r8**	MR	Valid	N.E.	Add *r8* to *r/m8.*
02 /r	ADD *r8, r/m8*	RM	Valid	Valid	Add *r/m8* to *r8.*
REX + 02 /r	ADD *r8**, *r/m8**	RM	Valid	N.E.	Add *r/m8* to *r8.*

*In 64-bit mode, r/m8 can not be encoded to access the following byte registers if a REX prefix is used: AH, BH, CH, DH.

Which Description Is Buggy?

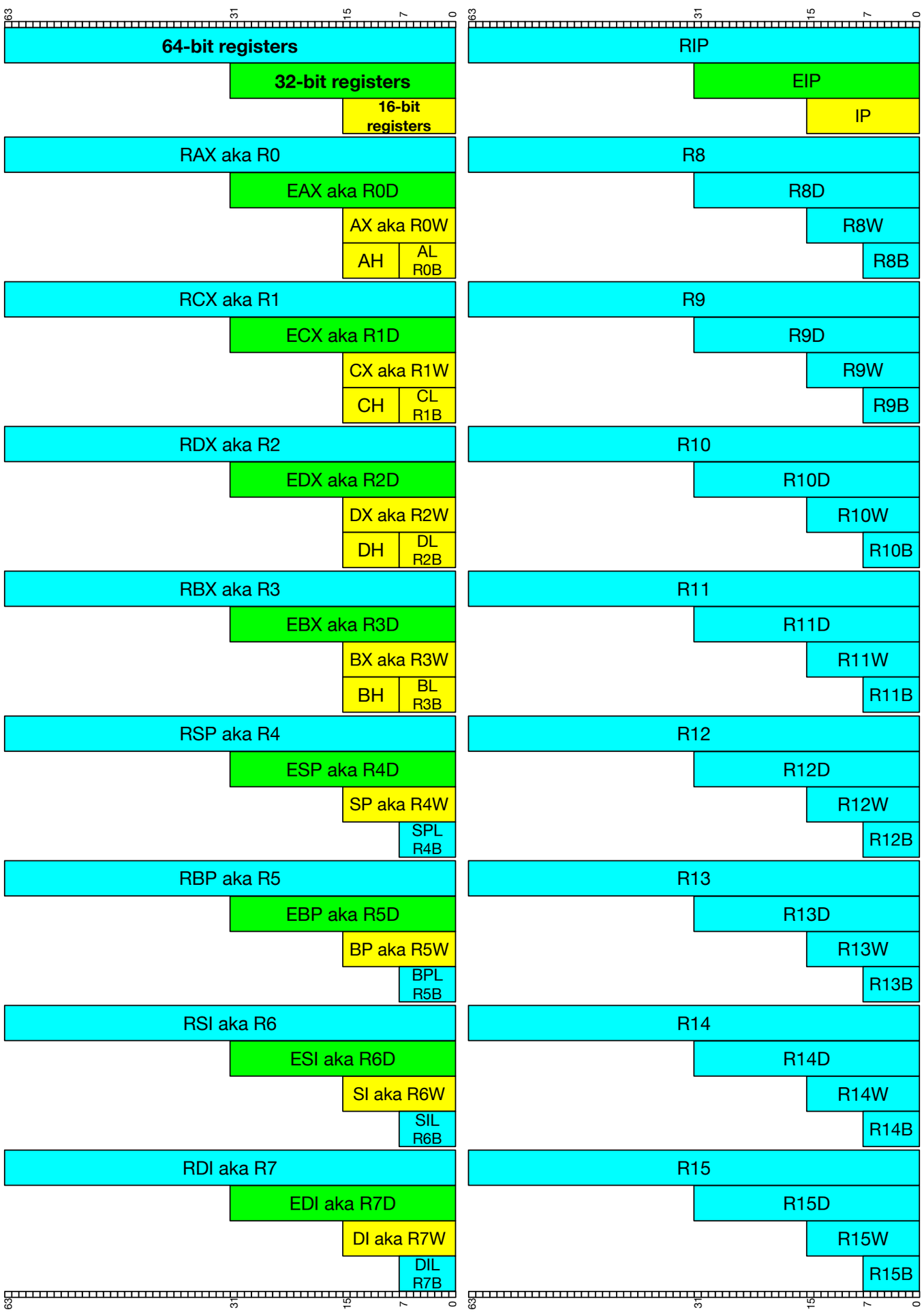
80 /4 *ib*	AND *r/m8, imm8*	MI	Valid	Valid	*r/m8* AND *imm8.*
REX + 80 /4 *ib*	AND *r/m8**, *imm8*	MI	Valid	N.E.	*r/m8* AND *imm8.*
20 /r	AND *r/m8, r8*	MR	Valid	Valid	*r/m8* AND *r8.*
REX + 20 /r	AND *r/m8**, *r8**	MR	Valid	N.E.	*r/m64* AND *r8 (sign-extended).*
22 /r	AND *r8, r/m8*	RM	Valid	Valid	*r8* AND *r/m8.*
REX + 22 /r	AND *r8**, *r/m8**	RM	Valid	N.E.	*r/m64* AND *r8 (sign-extended).*

Bugs!

80 /0 *ib*	ADD *r/m8, imm8*	MI	Valid	Valid	Add *imm8* to *r/m8.*
REX + 80 /0 *ib*	ADD *r/m8**, *imm8*	MI	Valid	N.E.	Add *sign*-extended *imm8* to *r/m8.*
00 /r	ADD *r/m8, r8*	MR	Valid	Valid	Add *r8* to *r/m8.*
REX + 00 /r	ADD *r/m8**, *r8**	MR	Valid	N.E.	Add *r8* to *r/m8.*
02 /r	ADD *r8, r/m8*	RM	Valid	Valid	Add *r/m8* to *r8.*
REX + 02 /r	ADD *r8**, *r/m8**	RM	Valid	N.E.	Add *r/m8* to *r8.*

* In 64-bit mode, r/m8 can not be encoded to access the following byte registers if a REX prefix is used: AH, BH, CH, DH.

The point of the "REX variants, is to allow 64 bit code to access 64-bit only byte registers like SPL, BPL, SIL, DIL, and R8L-R15L


63
31
15
7
0
64-bit registers
32-bit registers
16-bit registers
RAX aka R0
EAX aka R0D
AX aka R0W
AH
AL R0B
RCX aka R1
ECX aka R1D
CX aka R1W
CH
CL R1B
RDX aka R2
EDX aka R2D
DX aka R2W
DH
DL R2B
RBX aka R3
EBX aka R3D
BX aka R3W
BH
BL R3B
RSP aka R4
ESP aka R4D
SP aka R4W
SPL R4B
RBP aka R5
EBP aka R5D
BP aka R5W
BPL R5B
RSI aka R6
ESI aka R6D
SI aka R6W
SIL R6B
RDI aka R7
EDI aka R7D
DI aka R7W
DIL R7B
RIP
EIP
IP
R8
R8D
R8W
R8B
R9
R9D
R9W
R9B
R10
R10D
R10W
R10B
R11
R11D
R11W
R11B
R12
R12D
R12W
R12B
R13
R13D
R13W
R13B
R14
R14D
R14W
R14B
R15
R15D
R15W
R15B

AND Details

Description

Performs a bitwise AND operation on the destination (first) and source (second) operands and stores the result in the destination operand location. The source operand can be an immediate, a register, or a memory location; the destination operand can be a register or a memory location. (However, two memory operands cannot be used in one instruction.) Each bit of the result is set to 1 if both corresponding bits of the first and second operands are 1; otherwise, it is set to 0.

This instruction can be used with a LOCK prefix to allow the it to be executed atomically.

In 64-bit mode, the instruction's default operation size is 32 bits. Using a REX prefix in the form of REX.R permits access to additional registers (R8-R15). Using a REX prefix in the form of REX.W promotes operation to 64 bits. See the summary chart at the beginning of this section for encoding data and limits.

Operation

```
DEST := DEST AND SRC;
```

Flags Affected

The OF and CF flags are cleared; the SF, ZF, and PF flags are set according to the result. The state of the AF flag is undefined.

MORE!

ADD—Add

Opcode	Instruction	Op/ En	64-bit Mode	Compat/ Leg Mode	Description
04 *ib*	ADD AL, *imm8*	I	Valid	Valid	Add *imm8* to AL.
05 *iw*	ADD AX, *imm16*	I	Valid	Valid	Add *imm16* to AX.
05 *id*	ADD EAX, *imm32*	I	Valid	Valid	Add *imm32* to EAX.
REX.W + 05 *id*	ADD RAX, *imm32*	I	Valid	N.E.	Add *imm32 sign-extended to 64-bits* to RAX.
80 /0 *ib*	ADD *r/m8, imm8*	MI	Valid	Valid	Add *imm8* to *r/m8.*
REX + 80 /0 *ib*	ADD *r/m8**, *imm8*	MI	Valid	N.E.	Add *sign*-extended *imm8* to *r/m8.*
81 /0 *iw*	ADD *r/m16, imm16*	MI	Valid	Valid	Add *imm16* to *r/m16.*
81 /0 *id*	ADD *r/m32, imm32*	MI	Valid	Valid	Add *imm32* to *r/m32.*
REX.W + 81 /0 *id*	ADD *r/m64, imm32*	MI	Valid	N.E.	Add *imm32 sign-extended to 64-bits* to *r/m64.*
83 /0 *ib*	ADD *r/m16, imm8*	MI	Valid	Valid	Add *sign*-extended *imm8* to *r/m16.*
83 /0 *ib*	ADD *r/m32, imm8*	MI	Valid	Valid	Add *sign*-extended *imm8* to *r/m32.*
REX.W + 83 /0 *ib*	ADD *r/m64, imm8*	MI	Valid	N.E.	Add *sign*-extended *imm8* to *r/m64.*
00 /*r*	ADD *r/m8, r8*	MR	Valid	Valid	Add *r8* to *r/m8.*
REX + 00 /*r*	ADD *r/m8**, *r8**	MR	Valid	N.E.	Add *r8* to *r/m8.*
01 /*r*	ADD *r/m16, r16*	MR	Valid	Valid	Add *r16* to *r/m16.*
01 /*r*	ADD *r/m32, r32*	MR	Valid	Valid	Add *r32* to *r/m32.*
REX.W + 01 /*r*	ADD *r/m64, r64*	MR	Valid	N.E.	Add *r64* to *r/m64.*
02 /*r*	ADD *r8, r/m8*	RM	Valid	Valid	Add *r/m8* to *r8.*
REX + 02 /*r*	ADD *r8**, *r/m8**	RM	Valid	N.E.	Add *r/m8* to *r8.*
03 /*r*	ADD *r16, r/m16*	RM	Valid	Valid	Add *r/m16* to *r16.*
03 /*r*	ADD *r32, r/m32*	RM	Valid	Valid	Add *r/m32* to *r32.*
REX.W + 03 /*r*	ADD *r64, r/m64*	RM	Valid	N.E.	Add *r/m64* to *r64.*

ADD Details

Description

Adds the destination operand (first operand) and the source operand (second operand) and then stores the result in the destination operand. The destination operand can be a register or a memory location; the source operand can be an immediate, a register, or a memory location. (However, two memory operands cannot be used in one instruction.) When an immediate value is used as an operand, it is sign-extended to the length of the destination operand format.

The ADD instruction performs integer addition. It evaluates the result for both signed and unsigned integer operands and sets the CF and OF flags to indicate a carry (overflow) in the signed or unsigned result, respectively. The SF flag indicates the sign of the signed result.

This instruction can be used with a LOCK prefix to allow the instruction to be executed atomically.

In 64-bit mode, the instruction's default operation size is 32 bits. Using a REX prefix in the form of REX.R permits access to additional registers (R8-R15). Using a REX prefix in the form of REX.W promotes operation to 64 bits. See the summary chart at the beginning of this section for encoding data and limits.

Operation

```
DEST := DEST + SRC;
```

Flags Affected

The OF, SF, ZF, AF, CF, and PF flags are set according to the result.

Less Obvious

Less Obvious

- It turns out that for instructions that set status flags (e.g. arithmetic operations), the *hardware* just does the operation and sets flags as if the operands were **both unsigned and signed**

Less Obvious

- It turns out that for instructions that set status flags (e.g. arithmetic operations), the *hardware* just does the operation and sets flags as if the operands were **both unsigned and signed**
- Basically the hardware doesn't know or care about whether the humans are currently interpreting the bits as signed or unsigned. That's the compiler's problem to sort out.

Less Obvious

- It turns out that for instructions that set status flags (e.g. arithmetic operations), the *hardware* just does the operation and sets flags as if the operands were **both unsigned and signed**
- Basically the hardware doesn't know or care about whether the humans are currently interpreting the bits as signed or unsigned. That's the compiler's problem to sort out.
- The compiler must emit instructions which treat the bits as signed or unsigned based on what's specified in the high level language

MORE!

Jcc—Jump if Condition Is Met

Opcode	Instruction	Op/ En	64-Bit Mode	Compat/ Leg Mode	Description
77 *cb*	JA *rel8*	D	Valid	Valid	Jump short if above (CF=0 and ZF=0).
73 *cb*	JAE *rel8*	D	Valid	Valid	Jump short if above or equal (CF=0).
72 *cb*	JB *rel8*	D	Valid	Valid	Jump short if below (CF=1).
76 *cb*	JBE *rel8*	D	Valid	Valid	Jump short if below or equal (CF=1 or ZF=1).
72 *cb*	JC *rel8*	D	Valid	Valid	Jump short if carry (CF=1).
E3 *cb*	JCXZ *rel8*	D	N.E.	Valid	Jump short if CX register is 0.
E3 *cb*	JECXZ *rel8*	D	Valid	Valid	Jump short if ECX register is 0.
E3 *cb*	JRCXZ *rel8*	D	Valid	N.E.	Jump short if RCX register is 0.
74 *cb*	JE *rel8*	D	Valid	Valid	Jump short if equal (ZF=1).
7F *cb*	JG *rel8*	D	Valid	Valid	Jump short if greater (ZF=0 and SF=OF).
7D *cb*	JGE *rel8*	D	Valid	Valid	Jump short if greater or equal (SF=OF).
7C *cb*	JL *rel8*	D	Valid	Valid	Jump short if less (SF≠ OF).
7E *cb*	JLE *rel8*	D	Valid	Valid	Jump short if less or equal (ZF=1 or SF≠ OF).
76 *cb*	JNA *rel8*	D	Valid	Valid	Jump short if not above (CF=1 or ZF=1).
72 *cb*	JNAE *rel8*	D	Valid	Valid	Jump short if not above or equal (CF=1).
73 *cb*	JNB *rel8*	D	Valid	Valid	Jump short if not below (CF=0).
77 *cb*	JNBE *rel8*	D	Valid	Valid	Jump short if not below or equal (CF=0 and ZF=0).
73 *cb*	JNC *rel8*	D	Valid	Valid	Jump short if not carry (CF=0).
75 *cb*	JNE *rel8*	D	Valid	Valid	Jump short if not equal (ZF=0).
7E *cb*	JNG *rel8*	D	Valid	Valid	Jump short if not greater (ZF=1 or SF≠ OF).
7C *cb*	JNGE *rel8*	D	Valid	Valid	Jump short if not greater or equal (SF≠ OF).
7D *cb*	JNL *rel8*	D	Valid	Valid	Jump short if not less (SF=OF).

72 *cb*	JNAE *rel8*	D	Valid	Valid	Jump short if not above or equal (CF=1).
73 *cb*	JNB *rel8*	D	Valid	Valid	Jump short if not below (CF=0).
77 *cb*	JNBE *rel8*	D	Valid	Valid	Jump short if not below or equal (CF=0 and ZF=0).
73 *cb*	JNC *rel8*	D	Valid	Valid	Jump short if not carry (CF=0).
75 *cb*	JNE *rel8*	D	Valid	Valid	Jump short if not equal (ZF=0).
7E *cb*	JNG *rel8*	D	Valid	Valid	Jump short if not greater (ZF=1 or SF≠ OF).
7C *cb*	JNGE *rel8*	D	Valid	Valid	Jump short if not greater or equal (SF≠ OF).
7D *cb*	JNL *rel8*	D	Valid	Valid	Jump short if not less (SF=OF).
7F *cb*	JNLE *rel8*	D	Valid	Valid	Jump short if not less or equal (ZF=0 and SF=OF).
71 *cb*	JNO *rel8*	D	Valid	Valid	Jump short if not overflow (OF=0).
7B *cb*	JNP *rel8*	D	Valid	Valid	Jump short if not parity (PF=0).
79 *cb*	JNS *rel8*	D	Valid	Valid	Jump short if not sign (SF=0).
75 *cb*	JNZ *rel8*	D	Valid	Valid	Jump short if not zero (ZF=0).
70 *cb*	JO *rel8*	D	Valid	Valid	Jump short if overflow (OF=1).
7A *cb*	JP *rel8*	D	Valid	Valid	Jump short if parity (PF=1).
7A *cb*	JPE *rel8*	D	Valid	Valid	Jump short if parity even (PF=1).
7B *cb*	JPO *rel8*	D	Valid	Valid	Jump short if parity odd (PF=0).
78 *cb*	JS *rel8*	D	Valid	Valid	Jump short if sign (SF=1).
74 *cb*	JZ *rel8*	D	Valid	Valid	Jump short if zero (ZF = 1).
0F 87 *cw*	JA *rel16*	D	N.S.	Valid	Jump near if above (CF=0 and ZF=0). Not supported in 64-bit mode.
0F 87 *cd*	JA *rel32*	D	Valid	Valid	Jump near if above (CF=0 and ZF=0).
0F 83 *cw*	JAE *rel16*	D	N.S.	Valid	Jump near if above or equal (CF=0). Not supported in 64-bit mode.

Jcc Revisited

- If you look closely, you will see that there are multiple mnemonics for the same opcodes. The opcodes prove they're the same instructions
- Which mnemonic is displayed is disassembler-dependent

74 *cb*	JE *rel8*	D	Valid	Valid	Jump short if equal (ZF=1).
74 *cb*	JZ *rel8*	D	Valid	Valid	Jump short if zero (ZF = 1).

77 *cb*	JA *rel8*	D	Valid	Valid	Jump short if above (CF=0 and ZF=0).
77 *cb*	JNBE *rel8*	D	Valid	Valid	Jump short if not below or equal (CF=0 and ZF=0).

7C *cb*	JL *rel8*	D	Valid	Valid	Jump short if less (SF≠ OF).
7C *cb*	JNGE *rel8*	D	Valid	Valid	Jump short if not greater or equal (SF≠ OF).

Jcc Revisited

- A "short" *control flow displacement* is always specified by a signed 8-bit number
- A "near" *control flow displacement* could be a signed 16-bit (except when running in 64-bit mode) or signed 32-bit number

77 *cb*	JA *rel8*	D	Valid	Valid	Jump short if above (CF=0 and ZF=0).

0F 87 *cw*	JA *rel16*	D	N.S.	Valid	Jump near if above (CF=0 and ZF=0). Not supported in 64-bit mode.
0F 87 *cd*	JA *rel32*	D	Valid	Valid	Jump near if above (CF=0 and ZF=0).

Jcc Revisited

- The "Operation" section is now a little more complicated w/ its own conditional control flow pseudocode!
- This probably should be updated to have a "IF 64-bit Mode" like JMP does...

Operation

```
IF condition
    THEN
        tempEIP := EIP + SignExtend(DEST);
        IF OperandSize = 16
            THEN tempEIP := tempEIP AND 0000FFFFH;
        FI;
    IF tempEIP is not within code segment limit
        THEN #GP(0);
        ELSE EIP := tempEIP
    FI;
FI;
```

MORE!

CALL—Call Procedure

Opcode	Instruction	Op/ En	64-bit Mode	Compat/ Leg Mode	Description
E8 *cw*	CALL *rel16*	D	N.S.	Valid	Call near, relative, displacement relative to next instruction.
E8 *cd*	CALL *rel32*	D	Valid	Valid	Call near, relative, displacement relative to next instruction. 32-bit displacement sign extended to 64-bits in 64-bit mode.
FF /2	CALL *r/m16*	M	N.E.	Valid	Call near, absolute indirect, address given in *r/m16.*
FF /2	CALL *r/m32*	M	N.E.	Valid	Call near, absolute indirect, address given in *r/m32.*
FF /2	CALL *r/m64*	M	Valid	N.E.	Call near, absolute indirect, address given in *r/m*64.
9A *cd*	CALL *ptr16:16*	D	Invalid	Valid	Call far, absolute, address given in operand.
9A *cp*	CALL *ptr16:32*	D	Invalid	Valid	Call far, absolute, address given in operand.
FF /3	CALL *m16:16*	M	Valid	Valid	Call far, absolute indirect address given in *m16:16.* In 32-bit mode: if selector points to a gate, then RIP = 32-bit zero extended displacement taken from gate; else RIP = zero extended 16-bit offset from far pointer referenced in the instruction.
FF /3	CALL *m16:32*	M	Valid	Valid	In 64-bit mode: If selector points to a gate, then RIP = 64-bit displacement taken from gate; else RIP = zero extended 32-bit offset from far pointer referenced in the instruction.
REX.W FF /3	CALL *m16:64*	M	Valid	N.E.	In 64-bit mode: If selector points to a gate, then RIP = 64-bit displacement taken from gate; else RIP = 64-bit offset from far pointer referenced in the instruction.

CALL—Call Procedure

Opcode	Instruction	Op/ En	64-bit Mode	Compat/ Leg Mode	Description
E8 *cw*	CALL *rel16*	D	N.S.	Valid	Call near, relative displacement relative to next instruction.
E8 *cd*	CALL *rel32*	D	Valid	Valid	Call near, relative displacement relative to next instruction. 32-bit displacement sign extended to 64-bits in 64-bit mode.
FF /2	CALL *r/m16*	M	N.E.	Valid	Call near, absolute indirect address given in *r/m16.*
FF /2	CALL *r/m32*	M	N.E.	Valid	Call near, absolute indirect address given in *r/m32.*
FF /2	CALL *r/m64*	M	Valid	N.E.	Call near, absolute indirect address given in *r/m*64.

- There's no "call short"
- "Absolute indirect" form calculates an absolute address, using either just an address in a register, or an address calculated with the r/m64 "base + index*scale + displacement" form

9A *cd*	CALL *ptr16:16*	D	Invalid	Valid	Call far, absolute, address given in operand.
9A *cp*	CALL *ptr16:32*	D	Invalid	Valid	Call far, absolute, address given in operand.
FF /3	CALL *m16:16*	M	Valid	Valid	Call far, absolute indirect address given in *m16:16.* In 32-bit mode: if selector points to a gate, then RIP = 32-bit zero extended displacement taken from gate; else RIP = zero extended 16-bit offset from far pointer referenced in the instruction.
FF /3	CALL *m16:32*	M	Valid	Valid	In 64-bit mode: If selector points to a gate, then RIP = 64-bit displacement taken from gate; else RIP = zero extended 32-bit offset from far pointer referenced in the instruction.
REX.W FF /3	CALL *m16:64*	M	Valid	N.E.	In 64-bit mode: If selector points to a gate, then RIP = 64-bit displacement taken from gate; else RIP = 64-bit offset from far pointer referenced in the instruction.

- "Far" control flow transfer involves the specification of a "segment selector". We learn about those in Arch2001

Description

Saves procedure linking information on the stack and branches to the called procedure specified using the target operand. The target operand specifies the address of the first instruction in the called procedure. The operand can be an immediate value, a general-purpose register, or a memory location.

This instruction can be used to execute four types of calls:

- **Near Call** — A call to a procedure in the current code segment (the segment currently pointed to by the CS register), sometimes referred to as an intra-segment call.
- **Far Call** — A call to a procedure located in a different segment than the current code segment, sometimes referred to as an inter-segment call.
- **Inter-privilege-level far call** — A far call to a procedure in a segment at a different privilege level than that of the currently executing program or procedure.
- **Task switch** — A call to a procedure located in a different task.

- That's all Arch2001 stuff!

Because of the implications of the different distance variants, privilege transitions, different execution modes, CET, and, the pseudocode for the call instruction is about 9.75 pages long!

Operation

```
IF near call
    THEN IF near relative call
        THEN
            IF OperandSize = 64
                THEN
                    tempDEST := SignExtend(DEST); (* DEST is rel32 *)
                    tempRIP := RIP + tempDEST;
                    IF stack not large enough for a 8-byte return address
                        THEN #SS(0); FI;
                    Push(RIP);
                    IF ShadowStackEnabled(CPL) AND DEST != 0
                        ShadowStackPush8B(RIP);
                    FI;
                    RIP := tempRIP;
            FI;
            IF OperandSize = 32
                THEN
                    tempEIP := EIP + DEST; (* DEST is rel32 *)
                    IF tempEIP is not within code segment limit THEN #GP(0); FI;
                    IF stack not large enough for a 4-byte return address
                        THEN #SS(0); FI;
                    Push(EIP);
                    IF ShadowStackEnabled(CPL) AND DEST != 0
                        ShadowStackPush4B(EIP);
                    FI;
                    EIP := tempEIP;
            FI;
            IF OperandSize = 16
                THEN
                    tempEIP := (EIP + DEST) AND 0000FFFFH; (* DEST is rel16 *)
                    IF tempEIP is not within code segment limit THEN #GP(0); FI;
                    IF stack not large enough for a 2-byte return address
                        THEN #SS(0); FI;
                    Push(IP);
                    IF ShadowStackEnabled(CPL) AND DEST != 0
                        (* IP is zero extended and pushed as a 32 bit value on shadow stack *)
                        ShadowStackPush4B(IP);
                    FI;
                    EIP := tempEIP;
            FI;
        ELSE (* Near absolute call *)
            IF OperandSize = 64
```

MORE!

RET—Return from Procedure

Opcode*	Instruction	Op/ En	64-Bit Mode	Compat/ Leg Mode	Description
C3	RET	ZO	Valid	Valid	Near return to calling procedure.
CB	RET	ZO	Valid	Valid	Far return to calling procedure.
C2 *iw*	RET *imm16*	I	Valid	Valid	Near return to calling procedure and pop *imm16* bytes from stack.
CA *iw*	RET *imm16*	I	Valid	Valid	Far return to calling procedure and pop *imm16* bytes from stack.

- RET supports both "near" & "far" control flow transfer. But you'll almost only ever see the near return (0xC3)
- This also shows that second form of RET I mentioned where it can also add some number of bytes to the stack pointer
- About 10 pages of Operation pseudocode here too :-/

MORE!

JMP—Jump

Opcode	Instruction	Op/ En	64-Bit Mode	Compat/ Leg Mode	Description
EB *cb*	JMP *rel8*	D	Valid	Valid	Jump short, RIP = RIP + 8-bit displacement sign extended to 64-bits
E9 *cw*	JMP *rel16*	D	N.S.	Valid	Jump near, relative, displacement relative to next instruction. Not supported in 64-bit mode.
E9 *cd*	JMP *rel32*	D	Valid	Valid	Jump near, relative, RIP = RIP + 32-bit displacement sign extended to 64-bits
FF /4	JMP *r/m16*	M	N.S.	Valid	Jump near, absolute indirect, address = zero-extended *r/m16*. Not supported in 64-bit mode.
FF /4	JMP *r/m32*	M	N.S.	Valid	Jump near, absolute indirect, address given in *r/m32*. Not supported in 64-bit mode.
FF /4	JMP *r/m64*	M	Valid	N.E.	Jump near, absolute indirect, RIP = 64-Bit offset from register or memory

- The typical 8/16/32 byte relative displacements as well as absolute indirect
- ~5 pages of pseudocode. It's almost like control flow transfer is hard!

- Also supports the far displacement we don't know about yet

EA *cd*	JMP *ptr16:16*	D	Inv.	Valid	Jump far, absolute, address given in operand
EA *cp*	JMP *ptr16:32*	D	Inv.	Valid	Jump far, absolute, address given in operand
FF /5	JMP *m16:16*	D	Valid	Valid	Jump far, absolute indirect, address given in *m16:16*
FF /5	JMP *m16:32*	D	Valid	Valid	Jump far, absolute indirect, address given in *m16:32.*
REX.W FF /5	JMP *m16:64*	D	Valid	N.E.	Jump far, absolute indirect, address given in *m16:64*.

MORE!

CBW/CWDE/CDQE—Convert Byte to Word/Convert Word to Doubleword/Convert Doubleword to Quadword

Opcode	Instruction	Op/ En	64-bit Mode	Compat/ Leg Mode	Description
98	CBW	ZO	Valid	Valid	AX := sign-extend of AL.
98	CWDE	ZO	Valid	Valid	EAX := sign-extend of AX.
REX.W + 98	CDQE	ZO	Valid	N.E.	RAX := sign-extend of EAX.

```
IF OperandSize = 16 (* Instruction = CBW *)
    THEN
        AX := SignExtend(AL);
    ELSE IF (OperandSize = 32, Instruction = CWDE)
        EAX := SignExtend(AX); FI;
    ELSE (* 64-Bit Mode, OperandSize = 64, Instruction = CDQE*)
        RAX := SignExtend(EAX);
FI;
```

CWD/CDQ/CQO—Convert Word to Doubleword/Convert Doubleword to Quadword

Opcode	Instruction	Op/ En	64-Bit Mode	Compat/ Leg Mode	Description
99	CWD	ZO	Valid	Valid	DX:AX := sign-extend of AX.
99	CDQ	ZO	Valid	Valid	EDX:EAX := sign-extend of EAX.
REX.W + 99	CQO	ZO	Valid	N.E.	RDX:RAX:= sign-extend of RAX.

```
IF OperandSize = 16 (* CWD instruction *)
    THEN
        DX := SignExtend(AX);
    ELSE IF OperandSize = 32 (* CDQ instruction *)
        EDX := SignExtend(EAX); FI;
    ELSE IF 64-Bit Mode and OperandSize = 64 (* CQO instruction*)
        RDX := SignExtend(RAX); FI;
FI;
```

Ok. That's enough for now

Mini Exercise:

- Go change the MulDivExample.c to use a signed "long long" variable instead of "unsigned long long"
- Who doth appear, and why?
- Read and understand the code